

Tecnólogo em Sistemas para Internet

CENTRO POKÉMON



Céu Tavares, Dominique Nazareth, Dylan Gomes e Jéssica Freitas.

BRASÍLIA
2026



Céu Tavares, Dominique Nazareth, Dylan Gomes e Jéssica Freitas

Centro Pokémon

Análise de requisitos; DER - Normalizado até 3FN; Modelo Lógico - Normalizado até 3FN; Dicionário de dados; SQL - DDL do banco no MYSQL Workbench.

Trabalho realizado no curso técnico do
Instituto Federal de Brasília para o
componente curricular Banco de Dados.

Professor: Hugo do Carmo Mendes Cesar

Brasília
2026



1. ANÁLISE DE REQUISITOS

1.1 Introdução

1.1.1 Propósito do Documento

Baseado nos mangás, anime e jogos, a proposta é criar um banco de dados baseado em Pokémon. Para melhor entendimento dos dados e facilidades ao registrar os pokémons.

1.1.2 Escopo do Produto

O sistema deverá ser capaz de registrar treinadores e pokémons novos, consultar e atualizar dados já registrados no próprio e em outros pontos da rede de centros pokemon mantendo um histórico de consultas, relatórios de uso e possível troca de pokémons entre treinadores, tudo de acordo com as normas da PIA(Pokémon Inspection Agency).

1.1.3 Definições e Abreviações

PIA - Agência de Inspeção Pokémon (Pokémon Inspection Agency).

HP - Pontos de Vida (Health Points).

1.2 Descrição Geral

1.2.1 Perspectiva do Produto

O sistema deve permitir o acesso a uma base de dados com o objetivo de auxiliar o trabalho das enfermeiras pokémon pré, peri e pós tratamento. Ele deve permitir que dados sejam acessados, alterados e inseridos em todos os pontos da rede.

1.2.2. Funções do Produto

O sistema deve ter a capacidade de registrar novos pokémons e treinadores e seus respectivos dados, modificar dados já existentes e acessar o banco de dados de consultas anteriores se existentes.



1.2.3 Restrições Gerais

Para acessar o sistema, cada enfermeira deverá se identificar por meio de autenticação, toda alteração será gravada e guardada para documentação de acordo com os parâmetros dados pela Agência de Inspeção Pokémon.

1.2.4 Características do usuário

Os usuários do sistema serão enfermeiras formadas e licenciadas, que posteriormente serão treinadas no uso desse sistema.

1.2.5 Repositório do projeto

Para fins de auditoria, revisão técnica e transparência no desenvolvimento, todo o código-fonte DDL, o dicionário de dados e os scripts de inserção de dados estão hospedados e versionados publicamente.

O repositório pode ser acessado através do link:

github.com/Dylangomezz/Projeto-Banco-de-Bados-Centro-Pokemon

1.3 Requisitos

1.3.1 Requisitos Funcionais

RF.1: Registrar novos pokémons.

Registrar novos pokémons (Código, Nome, Espécie, Natureza, HP Total, Nível, Tipo Primário, Tipo Secundário e ID do Treinador).

RF.2: Registrar novos treinadores.

O Sistema deve ser capaz de registrar novos treinadores o que deve incluir: providenciar um código de identificação único, o seu nome, sua data de nascimento, telefone e seu endereço.



RF.3: Alterar Dados dos pokémons e seus treinadores.

O sistema também deve ser capaz de alterar as informações dos pokémons e treinadores caso necessário.

RF.4: Acessar registros de consultas e inserir novos.

O sistema deve ser capaz de inserir novas consultas contendo nelas: O código de identificação do(s) pokémon (s) sendo tratado(s), o código de identificação da enfermeira, o código de identificação o centro pokémon onde a consulta está sendo feita, a data/hora da consulta, o HP atual do pokémon, se ele está sob alguma condição e uma descrição do tratamento.

RF.5:Cadastrar novas enfermeiras.

Cadastrar novas enfermeiras, incluindo explicitamente o atributo Senha (armazenada como Hash) para controle de acesso ao sistema.

RF.6:Identificar o usuário.

O sistema devido a natureza sensível das suas informações só libera para uso após identificação de usuário por meio de login e senha. O login será o código de enfermeira do usuário e a senha será cadastrada por ela.

1.3.2 Requisitos não Funcionais

RNF.1: Segurança

O sistema deve garantir que apenas usuários autenticados tenham acesso às informações por meio de login e senha.

As senhas devem ser armazenadas utilizando técnicas de criptografia (ex: hash com salt).

O sistema deve encerrar automaticamente as sessões inativas após 10 minutos..

RNF.2: Desempenho

O sistema deve responder a operações de consulta em até 3 segundos para até 100 usuários simultâneos.

As operações de escrita (cadastro/atualização) devem ser concluídas em até 5 segundos.



RNF.3: Disponibilidade

O sistema deve estar disponível 24 horas por dia, 7 dias por semana, com no máximo 1 hora de indisponibilidade mensal (exceto manutenções programadas).

RNF.4: Usabilidade

O sistema deve possuir interface intuitiva, permitindo que um usuário novo consiga realizar operações básicas (cadastro e consulta) com no máximo 30 minutos de treinamento. As telas devem seguir um padrão visual consistente.

RNF.5: Integridade de dados

O sistema deve garantir consistência dos dados por meio de restrições de integridade (chaves primárias, estrangeiras e validações). Deve haver mecanismos de backup automático diário.

RNF.6: Auditoria

O sistema deve registrar todas as ações relevantes (inserções, alterações e exclusões), armazenando:

- usuário responsável
- data e hora
- tipo de operação

Os registros devem ser mantidos por no mínimo 5 anos.

RNF.7: Integração sistêmica

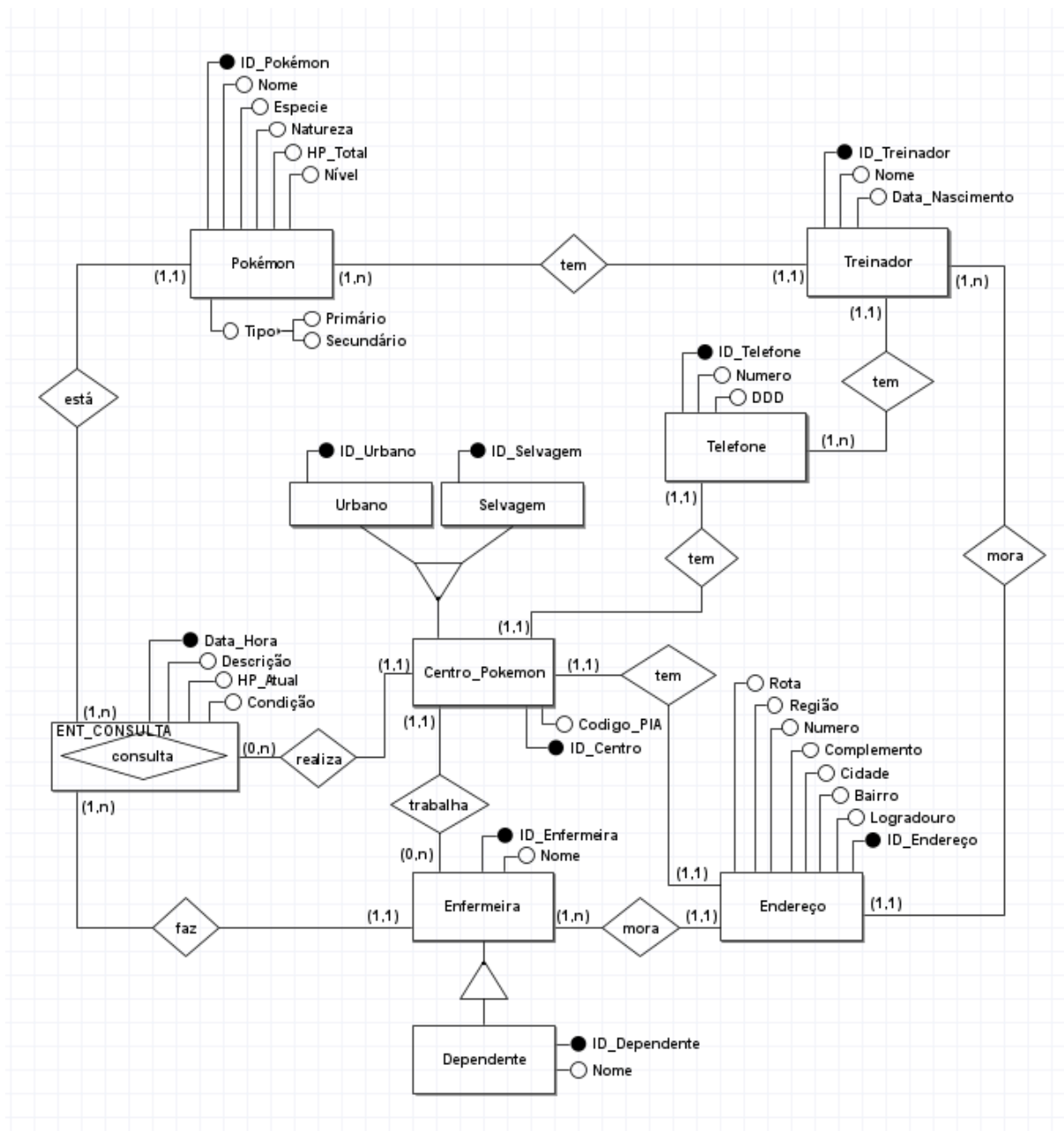
O sistema deve permitir a sincronização de dados entre centros Pokémon em tempo real ou com atraso máximo de 5 segundos. A comunicação deve ocorrer de forma segura (ex: HTTPS).

RNF.8: Manutenção

O sistema deve ser desenvolvido utilizando arquitetura modular, permitindo alterações sem impactar outros módulos. O código deve seguir padrões de desenvolvimento e possuir documentação mínima.



2. DIAGRAMA ENTIDADE RELACIONAMENTO



3.DESCRICÃO DE ENTIDADES E ATRIBUTOS

PK - Chave Primária	FK - Chave Estrangeira
---------------------	------------------------

3.1 Endereço:

Atributo	Tipo	Restrição	Descrição
<u>ID_Endereço</u>	INTEGER	PK	Identificador único do endereço.
Região	VARCHAR	NOT NULL	Nome da região.
Logradouro	VARCHAR	NOT NULL	Nome da rua, avenida ou praça.
Número	INTEGER	NOT NULL	Localização específica do imóvel.
Complemento	VARCHAR	NULL	Complemento do endereço.
Bairro	VARCHAR	NOT NULL	Nome do bairro.
Cidade	VARCHAR	NOT NULL	Nome da cidade.
Rota	INTEGER	NULL	Rota onde se localiza.

3.2 Treinador:

Atributo	Tipo	Restrição	Descrição
<u>ID_Treinador</u>	INTEGER	PK	Identificador único de um treinador Pokémon.
Nome	VARCHAR	NOT NULL	Nome do treinador Pokémon.
data_nascimento	DATA	NOT NULL	Data de nascimento do treinador Pokémon.
ID_Endereço	INTEGER	FK	Onde o Treinador mora.



3.3 Telefone:

Atributo	Tipo	Restrição	Descrição
<u>ID_Telefone</u>	INTEGER	PK	Identificador único do telefone.
Número	INTEGER	NOT NULL	Número do telefone.
DDD	INTEGER	NOT NULL	Código numérico de identificação regional.
ID_Treinador	INTEGER	FK	Dono do número.

3.4 Pokemon:

Atributo	Tipo	Restrição	Descrição
<u>ID_Pokémon</u>	INTEGER	PK	Identificador único de um Pokémon.
Nome	VARCHAR	NOT NULL	Nome do Pokémon.
Espécie	VARCHAR	NOT NULL	Espécie a qual o Pokémon pertence.
Natureza	VARCHAR	NOT NULL	A natureza do Pokémon.
HP_Total	INTEGER	NOT NULL	Quantidade de pontos de vida (<i>Hit Points</i>) do Pokémon.
Nível	INTEGER	NOT NULL	Nível do Pokémon.
Tipo_Primário	VARCHAR	NOT NULL	Tipo elemental do Pokémon.
Tipo_Secundário	VARCHAR	NULL	Tipo elemental secundário do Pokémon.
ID_Treinador	INTEGER	FK	Dono do pokemon.

3.5 Centro Pokémon:

Atributo	Tipo	Restrição	Descrição
<u>ID_Centro</u>	INTEGER	PK	Identificador único de um centro Pokémon.



Código_PIA	INTEGER	NOT NULL	Identificador do agente da PIA que aprovou o funcionamento do centro.
ID_Endereço	INTEGER	FK	Referência à localização geográfica e física do Centro Pokémon.

3.6 Urbano:

Atributo	Tipo	Restrição	Descrição
<u>ID_Centro</u>	INTEGER	PK/FK	Identificador do Centro Pokémon Urbano. Atua como Chave Primária e Estrangeira simultaneamente, herdando da tabela mãe centro pokemon.

3.7 Selvagem:

Atributo	Tipo	Restrição	Descrição
<u>ID_Centro</u>	INTEGER	PK/FK	Identificador do Centro Pokémon Selvagem. Atua como Chave Primária e Estrangeira simultaneamente, herdando da tabela mãe centro pokemon.

3.8 Enfermeira:

Atributo	Tipo	Restrição	Descrição
<u>ID_enfermeira</u>	INTEGER	PK	Identificador único de uma enfermeira.
Nome	VARCHAR	NOT NULL	Nome da enfermeira.
Senha_Hash	VARCHAR	NOT NULL	Senha criptografada por algoritmo de hash para autenticação segura no sistema.
ID_Centro	INTEGER	FK	Centro onde a enfermeira trabalha.



ID_Endereço	INTEGER	FK	Onde a enfermeira mora.
-------------	---------	----	-------------------------

3.9 Dependente:

Atributo	Tipo	Restrição	Descrição
<u>ID_Dependente</u>	INTEGER	PK	Data e hora de realização da consulta.
Nome	VARCHAR	NOT NULL	Nome do dependente
ID_Enfermeira	INTEGER	FK	Identificador da enfermeira que ele é dependente

3.10 Consulta (entidade associativa):

Atributo	Tipo	Restrição	Descrição
<u>Data_Hora</u>	INTEGER	PK	Data e hora de realização da consulta.
HP_Atual	INTEGER	NOT NULL	Quantidade atual de pontos de vida (<i>Hit Points</i>) do Pokémon no momento da respectiva consulta.
Condição	VARCHAR	NOT NULL	Condição que o Pokémon se encontra ao dar entrada no Centro.
ID_Pokémon	INTEGER	PK/FK	Pokemon atendido.
ID_Centro	INTEGER	PK/FK	Centro onde a consulta foi feita.
ID_Enfermeira	INTEGER	PK/FK	Enfermeira que fez a consulta.

4. REGRAS DE RELACIONAMENTO

Pokémon (1,n) — (1,1) Treinador

Um treinador pode ter vários pokémons, cada pokémon pertence a apenas um treinador.



Treinador (1,n) — (1,1) Endereço

Um treinador pode ter vários endereços e no mínimo um, cada endereço pertence a um treinador.

Treinador (1,1) — (1,n) Telefone

Um treinador cadastrado pode ter um ou mais telefones e o telefone só pode pertencer a um treinador.

Centro pokémon (1,1) — (1,1) Telefone

Um centro pokémon pode ter um único telefone e esse telefone só pode pertencer a um único centro pokémon.

Centro Pokémon (1,1) — (1,1) Endereço

Um centro pokémon pode ter um único endereço é esse endereço só pode pertencer a um único centro pokémon.

Centro Pokémon (1,1) — (1,1) Enfermeira

Uma enfermeira trabalha em um único centro pokémon e um centro pokémon pode trabalhar nenhuma ou várias enfermeiras.

Centro Pokémon (1,n) – Especialização – (1,1) Urbano

Urbano é uma especialização de um centro pokémon.

Centro Pokémon (1,n) – Especialização – (1,1) Selvagem

Selvagem é uma especialização de centro pokémon.

Centro Pokémon (1,1) — (0,n) Consulta

Uma consulta pode ser feita em um único centro pokémon e um centro pokémon pode fazer nenhum ou muitas consultas.

Enfermeira (1,n) — (1,1) Endereço

Uma enfermeira pode ter um único endereço registrado o endereço pode pertencer a no mínimo uma ou mais enfermeiras.

Enfermeira (1,1) – Especialização – (1,n) Dependente

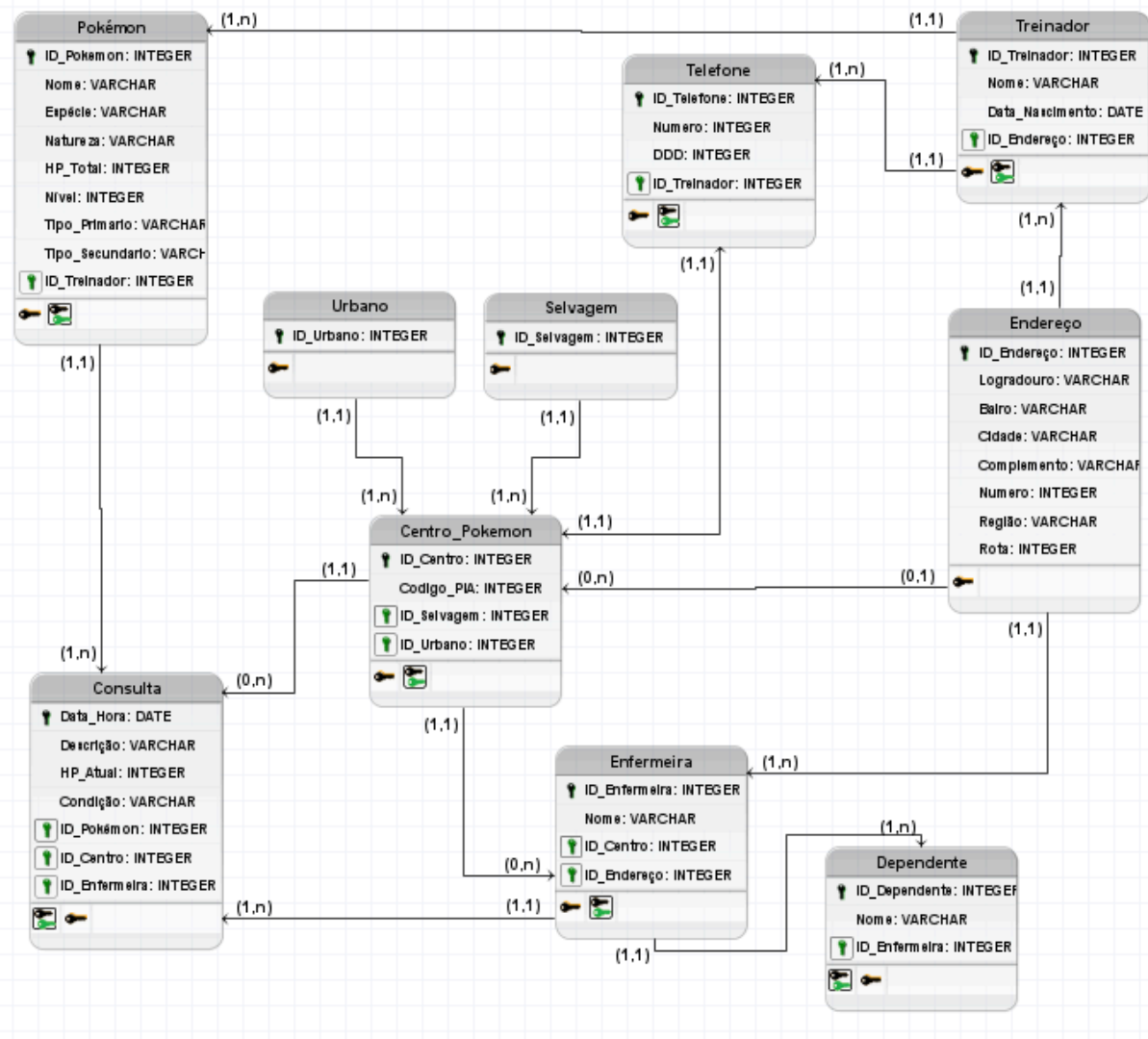
Dependente é uma especialização de enfermeira.

Pokémon (1,1) — (1,n) Consulta

Um pokémon pode estar em uma ou várias consultas, cada consulta é de um único pokémon.



5. MODELO RELACIONAL



tbEndereco (**id_endereco**: INTEGER, logradouro: VARCHAR(80), numero: INTEGER, complemento: VARCHAR(80), bairro: VARCHAR(80), cidade: VARCHAR(80), regiao: VARCHAR(80), rota: INTEGER)

tbTreinador (**id_treinador**: INTEGER, nome: VARCHAR(80), data_nascimento: DATE, *id_endereco*: INTEGER)
id_endereco referência **tbEndereco**

tbTelefone (**id_telefone**: INTEGER, numero: INTEGER, ddd: INTEGER, *id_treinador*: INTEGER)
id_treinador referência **tbTreinador**



tbPokemon (**id_pokemon**: INTEGER, nome: VARCHAR(80), especie: VARCHAR(80), natureza: VARCHAR(80), hp_total: INTEGER, nivel: INTEGER, tipo_primario: VARCHAR(80), tipo_secundario: VARCHAR(80), **id_treinador**: INTEGER)
id_treinador referência **tbTreinador**

tbCentro_Pokemon (**id_centro**: INTEGER, codigo_pia: INTEGER, **id_endereco**: INTEGER)
id_endereco referência **tbEndereco**

tbUrbano (**id_centro**: INTEGER)
id_centro atua como Chave Primária e Chave Estrangeira simultaneamente, referenciando **tbCentro_Pokemon**

tbSelvagem (**id_centro**: INTEGER)
id_centro atua como Chave Primária e Chave Estrangeira simultaneamente, referenciando **tbCentro_Pokemon**

tbEnfermeira (**id_enfermeira**: INTEGER, nome: VARCHAR(80), senha_hash: VARCHAR(80), **id_centro**: INTEGER, **id_endereco**: INTEGER)
id_centro referência **tbCentro_Pokemon**
id_endereco referência **tbEndereco**

tbDependente (**id_dependente**: INTEGER, nome: VARCHAR(80), **id_enfermeira**: INTEGER)
id_enfermeira referência **tbEnfermeira**

tbConsulta (**data_hora**: DATETIME, **id_pokemon**: INTEGER, **id_centro**: INTEGER, **id_enfermeira**: INTEGER, hp_atual: INTEGER, condicao: VARCHAR(80), descricao: VARCHAR(80))
id_pokemon referência **tbPokemon**
id_centro referência **tbCentro_Pokemon**
id_enfermeira referência **tbEnfermeira**



6. SQL

```
CREATE DATABASE IF NOT EXISTS db_centro_pokemon;
USE db_centro_pokemon;

-- Tabela endereço
create table if not exists tbEndereco (
id_endereco INT NOT NULL AUTO_INCREMENT,
regiao VARCHAR(80) NOT NULL,
logradouro VARCHAR(80) NOT NULL,
numero INT NOT NULL,
complemento VARCHAR(80) NULL,
bairro VARCHAR(80) NOT NULL,
cidade VARCHAR(80) NOT NULL,
rota VARCHAR(80) NULL,
PRIMARY KEY(id_endereco)
) ENGINE = InnoDB;

-- Tabela treinador
CREATE TABLE IF NOT EXISTS tbTreinador(
id_treinador INT NOT NULL AUTO_INCREMENT,
nome VARCHAR(80) NOT NULL,
data_nascimento DATE NOT NULL,
id_endereco INT NOT NULL,
PRIMARY KEY(id_treinador),
CONSTRAINT fk_treinador_endereco
    FOREIGN KEY (id_endereco)
    REFERENCES tbendereco (id_endereco)
    ON DELETE CASCADE
    ON UPDATE CASCADE
)ENGINE = InnoDB;

-- Tabela telefone
CREATE TABLE IF NOT EXISTS tbTelefone(
id_telefone INT NOT NULL AUTO_INCREMENT,
numero INT NOT NULL,
DDD INT NOT NULL,
id_treinador INT NOT NULL,
PRIMARY KEY (id_telefone),
CONSTRAINT fk_telefone_treinador
    FOREIGN KEY (id_treinador)
    REFERENCES tbtreinador (id_treinador)
```



```

        ON DELETE CASCADE
        ON UPDATE CASCADE
    )ENGINE = InnoDB;

-- Tabela pokemon
CREATE TABLE IF NOT EXISTS tbPokemon(
    id_pokemon INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(80) NOT NULL,
    especie VARCHAR(80) NOT NULL,
    natureza VARCHAR(80) NOT NULL,
    hp_total INT NOT NULL,
    nivel INT NOT NULL,
    tipo_primario VARCHAR(80) NOT NULL,
    tipo_secundario VARCHAR(80) NULL,
    id_treinador INT NOT NULL,
    PRIMARY KEY (id_pokemon),
    CONSTRAINT fk_pokemon_treinador
        FOREIGN KEY (id_treinador)
        REFERENCES tbtreinador (id_treinador)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    )ENGINE = InnoDB;

-- Tabela centro pokemon
CREATE TABLE IF NOT EXISTS tbCentro_Pokemon(
    id_centro INT NOT NULL AUTO_INCREMENT,
    codigo_pia INT NOT NULL,
    id_endereco INT NOT NULL,
    PRIMARY KEY (id_centro),
    CONSTRAINT fk_centro_endereco
        FOREIGN KEY (id_centro)
        REFERENCES tbEndereco (id_endereco)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    )ENGINE = InnoDB;

-- Tabela urbano
CREATE TABLE IF NOT EXISTS tbUrbano(
    id_centro INT NOT NULL,
    PRIMARY KEY (id_centro),
    CONSTRAINT fk_urbano_centro
        FOREIGN KEY (id_centro)

```



```

        REFERENCES tbcentro_pokemon (id_centro)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    )ENGINE = InnoDB;

-- Tabela selvagem
CREATE TABLE IF NOT EXISTS tbSelvagem(
    id_centro INT NOT NULL,
    PRIMARY KEY (id_centro),
    CONSTRAINT fk_selvagem_centro
        FOREIGN KEY (id_centro)
        REFERENCES tbcentro_pokemon (id_centro)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    )ENGINE = InnoDB;

-- Tabela enfermeira
CREATE TABLE IF NOT EXISTS tbEnfermeira(
    id_enfermeira INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(80) NOT NULL,
    senha_hash VARCHAR(80) NOT NULL,
    id_centro INT NOT NULL,
    id_endereco INT NOT NULL,
    PRIMARY KEY (id_enfermeira),
    CONSTRAINT fk_enfermeira_centro
        FOREIGN KEY (id_centro)
        REFERENCES tbcentro_pokemon (id_centro)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_enfermeira_endereco
        FOREIGN KEY (id_endereco)
        REFERENCES tbendereco (id_endereco)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    )ENGINE = InnoDB;

-- Tabela dependente
CREATE TABLE IF NOT EXISTS tbDependente (
    id_dependente INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(80) NOT NULL,
    id_enfermeira INT NOT NULL,
    PRIMARY KEY (id_dependente),

```



```

CONSTRAINT fk_dependente_enfermeira
    FOREIGN KEY (id_enfermeira)
    REFERENCES tbEnfermeira (id_enfermeira)
    ON DELETE CASCADE
    ON UPDATE CASCADE
) ENGINE = InnoDB;

-- Tabela associativa consulta
CREATE TABLE IF NOT EXISTS tbConsulta (
    data_hora DATETIME NOT NULL,
    descricao VARCHAR(1000) NOT NULL,
    hp_atual INT NOT NULL,
    condicao VARCHAR(80) NOT NULL,
    id_pokemon INT NOT NULL,
    id_centro INT NOT NULL,
    id_enfermeira INT NOT NULL,
    PRIMARY KEY (data_hora, id_pokemon, id_centro, id_enfermeira),
    CONSTRAINT fk_consulta_pokemon
        FOREIGN KEY (id_pokemon)
        REFERENCES tbPokemon (id_pokemon)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_consulta_centro
        FOREIGN KEY (id_centro)
        REFERENCES tbCentro_Pokemon (id_centro)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_consulta_enfermeira
        FOREIGN KEY (id_enfermeira)
        REFERENCES tbEnfermeira (id_enfermeira)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
) ENGINE = InnoDB;

```



```

-- insert
-- Tabela endereco
INSERT INTO tbEndereco (logradouro, bairro, cidade, complemento,
numero, regioao, rota)
VALUES ('Avenida Pallet Town,
Numero 10',
'Bairro do Centro',
'Cidade de Pallet',
'Proximo ao laboratorio do Professor Oak',
10,
'Regiao de Kanto',
NULL);

INSERT INTO tbEndereco (logradouro, bairro, cidade, complemento,
numero, regioao, rota)
VALUES ('Estrada da Rota Um, Sem Numero',
'Bairro da Floresta',
'Cidade de Viridian Outskirts',
'Kilometro 4 da via principal',
0,
'Regiao de Kanto',
1);

INSERT INTO tbEndereco (logradouro, bairro, cidade, complemento,
numero, regioao, rota)
VALUES ('Praça da Alvorada, Numero 500', 'Bairro Comercial',
'Cidade de Celadon', 'Ao lado do Shopping de Celadon', 500,
'Regiao de Kanto', NULL);

INSERT INTO tbEndereco (logradouro, bairro, cidade, complemento,
numero, regioao, rota)
VALUES ('Trilha da Floresta de Viridian', 'Área de Preservacao',
'Floresta Fechada', 'Proximo ao lago dos Gyarados', 0, 'Regiao de
Kanto', 2);

-- Tabela treinador
INSERT INTO tbTreinador (nome, data_nascimento, id_endereco)
VALUES ('Ash Ketchum da Cidade de Pallet',
'1997-04-01',
1);

INSERT INTO tbTreinador (nome, data_nascimento, id_endereco)

```



```

VALUES ('Gary Oak da Cidade de Pallet', '1997-04-02', 1);

-- Tabela telefone
INSERT INTO tbTelefone (numero, ddd, id_treinador)
VALUES (988887777, 11, 1);

INSERT INTO tbTelefone (numero, ddd, id_treinador)
VALUES (977776666, 11, 2);

-- Tabela pokemon
INSERT INTO tbPokemon (nome, especie, natureza, hp_total, nivel,
tipo_primario, tipo_secundario, id_treinador)
VALUES ('Pikachu do Ash',
'Mouse Pokemon', 'Jolly', 120, 25, 'Electric', NULL, 1);

INSERT INTO tbPokemon (nome, especie, natureza, hp_total, nivel,
tipo_primario, tipo_secundario, id_treinador)
VALUES ('Charizard do Ash', 'Flame Pokemon', 'Adamant', 280, 50,
'Fire', 'Flying', 1);

-- Tabela centro pokemon
INSERT INTO tbCentro_Pokemon (codigo_pia, id_endereco)
VALUES (1001, 1);

INSERT INTO tbCentro_Pokemon (codigo_pia, id_endereco)
VALUES (1002, 2);

INSERT INTO tbCentro_Pokemon (id_centro, codigo_pia, id_endereco)
VALUES (3, 1003, 3);

INSERT INTO tbCentro_Pokemon (id_centro, codigo_pia, id_endereco)
VALUES (4, 1004, 4);

-- Tabela urbano
INSERT INTO tbUrbano (id_centro) VALUES (1);
INSERT INTO tbUrbano (id_centro) VALUES (3);

-- Tabela selvagem
INSERT INTO tbSelvagem (id_centro) VALUES (2);
INSERT INTO tbSelvagem (id_centro) VALUES (4);

-- Tabela enfermeira

```



```
INSERT INTO tbEnfermeira (nome, senha_hash, id_centro,
id_endereco)
VALUES ('Nurse Joy de Pallet Town',
'HashCriptografadoGeradoPeloSistemaParaAjoy12345678', 1, 1);

INSERT INTO tbEnfermeira (nome, senha_hash, id_centro,
id_endereco)
VALUES ('Nurse Joy de Viridian City',
'SenhaSeguraComAlgoritmoDeHashComSaltParaSeguranca99', 2, 2);

-- Tabela dependente
INSERT INTO tbDependente (nome, id_enfermeira)
VALUES ('Joyzinha Junior de Pallet', 1);

INSERT INTO tbDependente (nome, id_enfermeira)
VALUES ('Joyzinha Sobrinha de Viridian', 2);
```



Apêndices

